

SuccessionTaskGenerator_Test Class

ISTEST

Test class for SuccessionTaskGenerator

Author Salesforce

Date October 2025

Class Diagram

```
graph TD
    SuccessionTaskGenerator_Test["SuccessionTaskGenerator_Test"]:::mainApexClass
    click SuccessionTaskGenerator_Test "/objects/SuccessionTaskGenerator_Test/"
    SuccessionTaskGenerator["SuccessionTaskGenerator"]:::apexClass
    click SuccessionTaskGenerator "/apex/SuccessionTaskGenerator/"

    SuccessionTaskGenerator_Test --> SuccessionTaskGenerator

classDef apexClass fill:#FFF4C2,stroke:#CCAA00,stroke-width:3px,rx:12px,ry:12px,shadow:drop,color:#333;
classDef apexTestClass fill:#F5F5F5,stroke:#999999,stroke-width:3px,rx:12px,ry:12px,shadow:drop,color:#333;
classDef mainApexClass fill:#FFB3B3,stroke:#A94442,stroke-width:4px,rx:14px,ry:14px,shadow:drop,color:#333,font-weight:bold;

linkStyle 0 stroke:#4C9F70,stroke-width:4px;
```

Apex Code

```
/**
 * @description Test class for SuccessionTaskGenerator
 * @author Salesforce
 * @date October 2025
 */
@isTest
private class SuccessionTaskGenerator_Test {

    @testSetup
    static void setup() {
        // Create Person Account (FSC)
        Id personAcctRecordTypeId =
Schema.SObjectType.Account.getRecordTypeInfoByDeveloperName()
        .get('PersonAccount').getRecordTypeId();

        Account testAccount = new Account(
```

```

        FirstName = 'Test',
        LastName = 'Successor',
        PersonEmail = 'successor@test.com',
        RecordTypeId = personAcctRecordTypeId
    );
    insert testAccount;

    // Get Estate Administration Record Type
    Id estateRecordTypeId =
Schema.SObjectType.Case.getRecordTypeInfoByDeveloperName()
    .get('EstateAdministration').getRecordTypeId();

    // Create test Case
    Case testCase = new Case(
        Subject = 'Test Succession Case',
        RecordTypeId = estateRecordTypeId,
        Type = 'Named Successor Enactment',
        AccountId = testAccount.Id,
        Status = 'New',
        Origin = 'Email'
    );
    insert testCase;
}

@isTest
static void testFinalGrantTasks() {
    Case testCase = [SELECT Id, CaseNumber FROM Case LIMIT 1];

    Test.startTest();

    // Trigger pathway task creation
    testCase.Pathway_Confirmed__c = 'Final Grant';
    testCase.Form_Completed_Date__c = Date.today();
    update testCase;

    Test.stopTest();

    // Verify 5 tasks were created
    List<Task> tasks = [
        SELECT Id, Subject, Priority, ActivityDate, Status
        FROM Task
        WHERE WhatId = :testCase.Id
        ORDER BY ActivityDate
    ];

    System.assertEquals(5, tasks.size(), 'Should create 5 Final Grant pathway
tasks');
    System.assertEquals('Review Grant Direction & Beneficiaries',
tasks[0].Subject);
    System.assertEquals('Coordinate Trading / Liquidation', tasks[1].Subject);
    System.assertEquals('High', tasks[0].Priority);
    System.assertEquals('Not Started', tasks[0].Status);

```

```

        // Verify due dates are staggered correctly (use >= for timezone tolerance)
        System.assert(tasks[0].ActivityDate >= Date.today().addDays(1), 'Task 1
should be at least 2 days from start');
        System.assert(tasks[1].ActivityDate >= Date.today().addDays(4), 'Task 2
should be at least 5 days from start');
        System.assert(tasks[2].ActivityDate >= Date.today().addDays(9), 'Task 3
should be at least 10 days from start');

        // Verify Chatter post was created
        List<FeedItem> posts = [SELECT Id, Body FROM FeedItem WHERE ParentId =
:testCase.Id];
        System.assertEquals(1, posts.size(), 'Should create 1 Chatter post');
        System.assert(posts[0].Body.contains('5 pathway tasks created'), 'Chatter
should mention task count');
    }

    @isTest
    static void testNewDAFTasks() {
        Case testCase = [SELECT Id FROM Case LIMIT 1];

        Test.startTest();

        testCase.Pathway_Confirmed__c = 'New DAF Account';
        testCase.Form_Completed_Date__c = Date.today();
        update testCase;

        Test.stopTest();

        // Verify 4 tasks were created
        List<Task> tasks = [
            SELECT Id, Subject, ActivityDate
            FROM Task
            WHERE WhatId = :testCase.Id
            ORDER BY ActivityDate
        ];

        System.assertEquals(4, tasks.size(), 'Should create 4 New DAF pathway
tasks');
        System.assertEquals('Review New Account Application', tasks[0].Subject);
        System.assertEquals('Process Transfer Authorization', tasks[1].Subject);
        System.assert(tasks[0].ActivityDate >= Date.today().addDays(1), 'First task
due date should be within range');
        System.assert(tasks[3].ActivityDate >= Date.today().addDays(17), 'Last task
due date should be within range');
    }

    @isTest
    static void testDisclaimTasks() {
        Case testCase = [SELECT Id FROM Case LIMIT 1];

        Test.startTest();

```

```

testCase.Pathway_Confirmed__c = 'Disclaim Assets';
testCase.Form_Completed_Date__c = Date.today();
update testCase;

Test.stopTest();

// Verify 4 tasks were created
List<Task> tasks = [
    SELECT Id, Subject, ActivityDate
    FROM Task
    WHERE WhatId = :testCase.Id
    ORDER BY ActivityDate
];

System.assertEquals(4, tasks.size(), 'Should create 4 Disclaim pathway
tasks');
System.assertEquals('Review Disclaimer Documentation', tasks[0].Subject);
System.assertEquals('Identify Alternate Distribution Path',
tasks[1].Subject);
System.assert(tasks[0].ActivityDate >= Date.today().addDays(2), 'First task
due date should be within range');
System.assert(tasks[3].ActivityDate >= Date.today().addDays(19), 'Last task
due date should be within range');
}

@isTest
static void testNoTasksCreated_NoPathway() {
    Case testCase = [SELECT Id FROM Case LIMIT 1];

    Test.startTest();

    // Update without setting pathway
    testCase.Status = 'In Progress';
    update testCase;

    Test.stopTest();

    List<Task> tasks = [SELECT Id FROM Task WHERE WhatId = :testCase.Id];
    System.assertEquals(0, tasks.size(), 'No tasks should be created without
pathway');
}

@isTest
static void testNoTasksCreated_PathwayUnchanged() {
    Case testCase = [SELECT Id FROM Case LIMIT 1];

    // Set pathway initially
    testCase.Pathway_Confirmed__c = 'Final Grant';
    testCase.Form_Completed_Date__c = Date.today();
    update testCase;

```

```

Test.startTest();

// Update without changing pathway
testCase.Status = 'In Progress';
update testCase;

Test.stopTest();

// Should still only have 5 tasks from first update
List<Task> tasks = [SELECT Id FROM Task WHERE WhatId = :testCase.Id];
System.assertEquals(5, tasks.size(), 'Should not create duplicate tasks');
}

@isTest
static void testBulkTaskCreation() {
    // Get Estate Administration Record Type
    Id estateRecordTypeId =
Schema.SObjectType.Case.getRecordTypeInfoByDeveloperName()
    .get('EstateAdministration').getRecordTypeId();

    // Create Person Account
    Id personAcctRecordTypeId =
Schema.SObjectType.Account.getRecordTypeInfoByDeveloperName()
    .get('PersonAccount').getRecordTypeId();

    Account testAccount = new Account(
        FirstName = 'Bulk',
        LastName = 'Test',
        PersonEmail = 'bulk@test.com',
        RecordTypeId = personAcctRecordTypeId
    );
    insert testAccount;

    // Create 200 test Cases
    List<Case> cases = new List<Case>();
    for (Integer i = 0; i < 200; i++) {
        cases.add(new Case(
            Subject = 'Bulk Test ' + i,
            RecordTypeId = estateRecordTypeId,
            Type = 'Named Successor Enactment',
            AccountId = testAccount.Id,
            Status = 'New',
            Origin = 'Email'
        ));
    }
    insert cases;

    Test.startTest();

    // Update all with pathways
    for (Integer i = 0; i < cases.size(); i++) {
        if (Math.mod(i, 3) == 0) {

```

```

        cases[i].Pathway_Confirmed__c = 'Final Grant';
    } else if (Math.mod(i, 3) == 1) {
        cases[i].Pathway_Confirmed__c = 'New DAF Account';
    } else {
        cases[i].Pathway_Confirmed__c = 'Disclaim Assets';
    }
    cases[i].Form_Completed_Date__c = Date.today();
}
update cases;

Test.stopTest();

// Verify tasks were created
List<Task> tasks = [SELECT Id FROM Task WHERE WhatId IN :cases];
System.assert(tasks.size() > 0, 'Bulk operation should create tasks');
// Total should be ~867 tasks (67*5 + 66*4 + 67*4)
}
}

```

Methods

setup()

TESTSETUP

Signature

```
private static void setup()
```

Return Type

void

testFinalGrantTasks()

ISTEST

Signature

```
private static void testFinalGrantTasks()
```

Return Type

void

testNewDAFTasks()

ISTEST

Signature

```
private static void testNewDAFTasks()
```

Return Type

void

testDisclaimTasks()

ISTEST

Signature

```
private static void testDisclaimTasks()
```

Return Type

void

testNoTasksCreated_NoPathway()

ISTEST

Signature

```
private static void testNoTasksCreated_NoPathway()
```

Return Type

void

testNoTasksCreated_PathwayUnchanged()

ISTEST

Signature

```
private static void testNoTasksCreated_PathwayUnchanged()
```

Return Type

void

testBulkTaskCreation()

ISTEST

Signature

```
private static void testBulkTaskCreation()
```

Return Type

void